Politechnika Warszawska

WYDZIAŁ ELEKTRYCZNY



Instytut Sterowania i Elektroniki Przemysłowej

Praca dyplomowa inżynierska

na kierunku Informatyka Stosowana

w specjalności Inżynieria Danych i Multimedia

Implementacja techniki trasowania cebulowego z wykorzystaniem mikrokontrolera z rodziny ESP32

Damian Ciaszczyk

numer albumu 331672

promotor

dr inż. Krzysztof Jackiewicz

WARSZAWA 2026

Implementacja techniki trasowania cebulowego z wykorzystaniem mikrokontrolera z rodziny ESP32

Streszczenie

Celem tej pracy dyplomowej była analiza oraz implementacja techniki trasowania cebulowego (ang. Onion Routing) na mikrokontrolerach z rodziny ESP32. W pracy zbadano możliwości wykorzystania tej techniki w urządzeniach wbudowanych oraz opracowano biblioteki w języku C umożliwiające ESP32 pełnienie funkcji klientów oraz przekaźników. Prototypy zostały zaimplementowane w formie szkiców (ang. sketch) arduino, natomiast końcowe rozwiązanie w oparciu o zadania systemu FreeRTOS. Ocenie podległ także wpływ zastosowania trasowania cebulowego na wydajność komunikacji, w szczególności na opóźnienia w dostarczaniu wiadomości, szybkość transmisji oraz wykorzystanie zasobów obliczeniowych i pamięciowych mikrokontrolerów. W pracy przeanalizowano ograniczenia implementacyjne związane z koniecznością ustanawiania tras komunikacyjnych, przechowywania informacji o przekaźnikach oraz wielokrotnego szyfrowania i odszyfrowywania przesyłanych danych. Ponadto zbadano potencjalne słabości rozwiązania, takie jak analiza czasowa ruchu sieciowego czy ryzyko związane z węzłem wyjściowym w przypadku braku dodatkowego szyfrowania end-to-end.

Na potrzeby tej pracy inżynierskiej powstały prototypy prezentujące komunikację standardowymi protokołami http/https, jak i stopniowe wprowadzanie techniki trasowania cebulowego celem zaprezentowania efektu który zapewnia w kwestii przesłaniania informacji o uczestnikach komunikacji. Dodatkowo, powstały proste kontenery realizację funkcje przekaźników. [TODO streszczenie wyników] Końcowo, powstały programy w C realizujące technikę trasowania cebulowego jako klient jak i przekaźnik na mikrokontrolerach z rodziny ESP32, oraz analizy identyfikujące sensowne zastosowania dla techniki trasowania cebulowego na tych mikrokontrolerach. Analizy wydajnościowe rozwiązania sugerują to i owo.

Wnioski [TODO]

Słowa kluczowe: ESP32, trasowanie cebulowe, szyfrowanie komunikacji, protokoły sieciowe

Implementation of the Onion routing technique using an ESP32 microcontroller

Abstract

Keywords: ESP32, Onion routing, encrypting communication, network protocols

Spis treści

1 Wstęp **9**
1.1 Struktura pracy 10

2 analiza wymagań **13**
2.1 Charakterystyka trasowania cebulowego w kontekście mikrokontrolerów 13
2.2 Wymagania funkcjonalne 14
2.3 Wymagania niefunkcjonalne 14
2.4 Metody spełnienia założeń trasowania cebulowego na ESP32 15
2.5 Podsumowanie analizy wymagań 16

3 prototypy teorytyczne **17**

4 projekt rozwiązania **19**

5 implementacja **21**

6 testy **23**

7 Podsumowanie **25**

Rozdział 1

Wstęp

Urządzenia Internetu Rzeczy (IoT) z roku na rok produkują oraz przesyłają coraz więcej danych. Wynika to z coraz powszechniejszego wykorzystania ich do zbierania statystyk oraz regulowania działania wszelkich struktur oraz mechanizmów. Ze względu na ograniczone zasoby sprzętowe urządzeń IoT, ich funkcje również są projektowane jak najprościej, w tym warstwa komunikacyjna. Przeważnie stosują standardowe protokoły komunikacyjne, wykorzystanie których w większości sytuacji jest adekwatnym rozwiązaniem. Jednak w przypadku urządzeń wbudowanych przesyłających szczególnie wrażliwe dane, poza standardowym szyfrowaniem przesyłanej wiadomości oferowanego na przykład przez TLS (ang. Transport Layer Security), pożądane może też być ukrycie z kim odbywa się sama komunikacja. Takie rozwiązanie zapewnia technika trasowania cebulowego (ang. Onion routing), z najbardziej znaną implementacją realizowaną przez protokoły sieci nakładkowej Tor. Sama technika trasowania cebulowego wzięła nazwę ze względu na szyfrowanie przesyłanej wiadomości kilkukrotnie, w warstwach nałożonych jedna na drugą, co przypomina swoją budową cebulę. Aby przesłać wiadomość tą techniką nadawca najpierw wybiera kilka przekaźników (ang. relays) tworząc z nich obwód (ang. circuit) przez który wiadomość zostanie przesłana. Następnie wiadomość jest szyfrowana tak, aby każdy kolejny przekaźnik mógł odszyfrować po jednej warstwie z danej wiadomości aż do przesłania odszyfrowanych danych przez ostatni przekaźnik do odbiorcy. W ten sposób tylko nadawca zna całą trasę wiadomości, natomiast przekaźniki znają co najwyżej odbiorcę albo nadawcę, nigdy obu na raz. Sam odbiorca również widzi wiadomość tak jakby została nadana przez ostatni przekaźnik, nie nadawcę. Dla wiadomości zwrotnej można odwrócić kolejność, gdzie z kolei przekaźniki szyfrują warstwa po warstwie całą wiadomość, tak aby tylko oryginalny nadawca mógł odszyfrować całą odpowiedź. Implementacja techniki trasowania cebulowego na urządzeniach o ograniczonych zasobach, takich jak mikrokontrolery, nie jest standardem. Implementacje tej techniki na urządzeniach wbudowanych są rzadkie, oraz nieutrzymywane. Wynika to z niewielkiego obszaru gdzie poświęcenie zasobów mikrokontrolera w tym celu ma sens, obszaru jednak istniejącego. Mikrokontrolery z rodziny ESP32 stanowią idealny punkt startowy do wprowadzenia takiej implementacji dzięki ich wbudowanemu modułowi Wi-Fi oraz sprzsętowych akceleratorów algorytmów AES, SHA, RSA oraz ECC. Wykorzystanie urządzeń z rodziny ESP32 z trasowaniem cebulowym nie jest jednak

dokładnie przebadane ani udokumentowane, a obciążenie które ta technika wprowadza nie zostało zmierzone.

Celem pracy jest zaprojektowanie, implementacja oraz analiza obciążenia techniki trasowania cebulowego z wykorzystaniem mikrokontrolera ESP32 jako klienta oraz lekkiego przekaźnika. Praca ma odpowiedzieć na pytanie, w jakich scenariuszach IoT takie rozwiązanie jest praktycznie wykonalne oraz sensowne, a w jakich jego wady przeważają nad zaletami.

Zakres pracy obejmuje:

- Przegląd literatury,
- Określenie wymagań funkcjonalnych spełniających założenia trasowania cebulowego,
- Projekt i wykonanie prostej sieci z nałożoną techniką trasowania cebulowego oraz monitorowaniem przesyłanych pakietów,
- Przygotowanie programów pozwalających na komunikację po sieci nakładkowej z wyjściem na zarówno węzeł należący do sieci, jak i węzeł zewnętrzny,
- Testy zaimplementowanego rozwiązania na fizycznym układzie złożonym z sieci mikrokontrolerów,
- Podsumowanie i wnioski.

Część praktyczna została zrealizowana z wykorzystaniem płytek deweloperskich bazujących na ESP32-S3-WROOM-1 oraz środowiska Arduino IDE. Kryptografia realizowana jest za pomocą sprzętowych akceleratorów ESP32 oraz bibliotek dostosowanych do systemów wbudowanych (mbedtls/aes.h na przykład, do rozwinięcia, opcja wyłączonego ADC?). Zaimplementowane rozwiązanie zostało porównane z różnymi konfiguracjami bardziej standardowych rozwiązań (biblioteka httpclient.h zapewnia http i https, alternatywnie można porównać z implementacją tls, ale https to samo używa).

1.1 Struktura pracy

Praca składa się z rozdziałów. Ilu? Nie wiem. Planuję te:

- Rozdział 1 (niniejszy) - wprowadzenie w temat, opis problemu, cel i zakres pracy.
- Rozdział 2 - analiza wymagań: charakteryzacja techniki trasowania cebulowego, oraz przedstawienie metody spełnienia jej założeń.
- Rozdział 3 - prototypy teorytyczne: przedstawienie prototypów porównujących kluczowe różnice między trasowaniem cebulowym, a bardziej klasycznymi metodami przesyłania pakietów.
- Rozdział 4 - projekt rozwiązania: architektura sieci testowej, opis funkcjonalny API modułu trasowania oraz przekaźnika, zebranie metod realizujących trasowanie cebulowe w pełni.
- Rozdział 5 - implementacja: prezentacja zaimplementowanego trasowania jak i najważniejszych elementów rozwiązania.
- Rozdział 6 - testy: porównanie wydajnościowe z innymi metodami przesyłu pakietów, testy bezpieczeństwa kryptograficznego przesyłanych danych, analiza sensownych zastosowań rozwiązania.

- Rozdział 7 - podsumowanie: osiągnięte cele, ograniczenia, napotkane problemy.

Rozdział 2

analiza wymagań

Rozdział ten ma na celu określenie konkretnych wymagań których realizacja spełnia ogólne założenia wynikające z tematu pracy. W tym celu poza określeniem wymagań realizujących założenia trasowania cebulowego, należy także uwzględnić ograniczenia związane z implementacją na mikrokontrolerach. Za dobry punkt odniesienia służy dokumentacja protokołów sieci nakładkowej Tor, która także implementuje tą technikę, aczkolwiek w formie wielokrotnie przekraczającej możlwości urządzenia takiego jak ESP32.

2.1 Charakterystyka trasowania cebulowego w kontekście mikrokontrolerów

Trasowanie cebulowe w swojej klasycznej postaci, znanej z sieci Tor, zakłada istnienie przekaźników wielokrotnie bogatszych w zasoby niż prosty mikrokontroler, zdolnych do utrzymywania wielu równoczesnych obwodów i buforowania pakietów. Mikrokontrolery działają na o wiele bardziej ograniczonych zasobach: w przypadku wykorzystywanej płytki deweloperskiej 512 kB SRAM, 8 MB PSRAM oraz 16 MB pamięci Flash, a jej moc obliczeniowa jest wiele rzędów wielkości mniejsza niż w przypadku serwerów pełniących rolę przekaźników Tor. Uruchomienie na ESP32 pełnego demona Tor nie jest możliwe, ponieważ sam demon wymaga kilkaset razy więcej megabajtów pamięci operacyjnej niż mikrokontroler oferuje.

Oznacza to, że analiza wymagań nie może opierać się na samym skopiowianiu architektury Tor. Konieczne jest zidentyfikowanie minimalnego zestawu funkcji trasowania cebulowego, który jest niezbędny do zachowania jego podstawowych właściwości bezpieczeństwa, przy jednoczesnym dostosowaniu ich do ograniczeń platformy. W szczególności należy rozstrzygnąć, na jakich warunkach ESP32 może pełnić rolę przekaźnika.

2.2 Wymagania funkcjonalne

Wymagania funkcjonalne określają, jakie funkcje rozwiązanie musi realizować, aby można było mówić o działającej implementacji trasowania cebulowego.

WF-1. Zestawianie obwodu trasowania cebulowego. Implementacja musi umożliwiać wybór sekwencji przekaźników i stworzenie obwodu, przez który będą przekazywane wiadomości. Stworzenie obwodu obejmuje ustalenie metody szyfrowania z każdym przekaźnikiem z osobna, sekwencyjnie korzystając z poprzednich przekaźników do komunikacji z następnymi.

- WF-2. Warstwowe szyfrowanie wiadomości. Implementacja musi realizować szyfrowanie wiadomości w taki sposób, aby każdy kolejny przekaźnik mógł odszyfrować wyłącznie jedną warstwę, nie poznając przy tym pełnej trasy ani tożsamości nadawcy i odbiorcy jednocześnie.
- WF-3. Przekazywanie wiadomości przez przekaźniki. Implementacja musi umożliwiać przekazywanie zaszyfrowanej wiadomości kolejno przez wybrane węzły, przy czym każdy węzeł pośredniczący usuwa jedną warstwę szyfrowania i przekazuje wiadomość dalej zgodnie z informacją zawartą w odszyfrowanej warstwie.
- WF-4. Obsługa wiadomości zwrotnych. Implementacja musi zapewniać mechanizm umożliwiający odbiorcy przesłanie odpowiedzi do nadawcy bez ujawniania jego adresu, przy zachowaniu analogicznego warstwowego szyfrowania w kierunku powrotnym.
- WF-5. Działanie ESP32 w roli przekaźnika. Implementacja musi zapewniać możliwość odbierania wiadomości zaadresowanej do przekaźnika, odszyfrowania jednej warstwy i przekazania wiadomości do kolejnego węzła lub odbiorcy końcowego.
- WF-6. Integracja z siecią przekaźników. Implementacja musi umożliwiać operowanie przekaźnika w większej sieci, przynajmniej częściowo.
- WF-7. Konfiguracja i monitorowanie. Implementacja musi udostępniać podstawowe mechanizmy konfiguracji (np. wybór liczby przekaźników, trybu pracy) oraz monitorowania stanu obwodu i podstawowych metryk wydajnościowych.

2.3 Wymagania niefunkcjonalne

Wymagania niefunkcjonalne określają jak dobrze system ma realizować swoje funkcje, czyli jakie są ograniczenia jakościowe i ilościowe. W przypadku implementacji na ESP32 mają one charakter krytyczny, ponieważ to właśnie one decydują o dozwolonym zużyciu zasobów, oraz wymaganej prędkości przesyłu.

WNF-1. Ograniczenia pamięciowe. Implementacja musi mieścić się w połowie dostępnych zasobów ESP32: Dla wykorzystywanej płytki to 8 MB flash oraz 256 kB SRAM. Oznacza to konieczność starannego zarządzania pamięcią, unikania dynamicznej alokacji dużych struktur oraz implementacji minimalnej liczby funkcji i zadań realizujących założenia trasowania cebulowego.

WNF-2. Ograniczenia obliczeniowe i czasowe. Czas zestawienia obwodu oraz czas przetwarzania pojedynczej wiadomości muszą mieścić się w akceptowalnych granicach dla docelowych zastosowań

IoT. Należy przy tym wykorzystać sprzętowe akceleratory kryptograficzne ESP32 (AES, SHA) w celu ograniczenia narzutu obliczeniowego.

WNF-3. Bezpieczeństwo kryptograficzne. Zastosowane algorytmy i protokoły muszą zapewniać co najmniej taki poziom poufności i integralności, jaki jest wymagany dla wrażliwych danych w IoT. Warstwowe szyfrowanie musi być odporne na podstawowe ataki analizy ruchu, w zakresie wynikającym z ograniczeń platformy. Szyfrowanie musi więc być kryptograficznie bezpieczne, co najmniej równoważne 128-bitowemu poziomowi bezpieczeństwa (np. AES-128-GCM).

WNF-4. Niezawodność i stabilność. System powinien poprawnie obsługiwać sytuacje utraty łączności z przekaźnikiem, przekroczenia czasu oczekiwania na odpowiedź oraz ponownego zestawiania obwodu.

2.4 Metody spełnienia założeń trasowania cebulowego na ESP32

Sformułowane wymagania prowadzą do pytania o konkretne podejścia techniczne, które pozwolą zrealizować technikę trasowania cebulowego na ESP32, przy założeniu, że każde urządzenie może występować zarówno jako klient, jak i jako przekaźnik. Oznacza to, że implementacja musi obsługiwać co najmniej trzy zestawy funkcji: zestawianie i utrzymywanie obwodów po stronie klienta, odbieranie oraz odszyfrowywanie jednej warstwy i przekazywanie wiadomości po stronie przekaźnika, jak i uczestnictwo w ustalaniu obwodów po stronie przekaźnika. Poniżej przedstawiono wybrany kierunek implementacyjny.

Podstawowym rozwiązaniem jest zaprojektowanie rozwiązania opierającego się o pakiety o stałym rozmiarze. Dobrym formatem jest Sphinx, który jest przystosowany do takiej komunikacji. Możliwe także jest zastosowanie uproszczonego formatu pakietów, gdyby pakiety sphinx okazały się za ciężkie dla mikrokontrolerów. Ponieważ ESP32 ma pełnić również rolę przekaźnika, analiza wymagań musi objąć model współbieżności i gospodarkę pamięcią. Przekaźnik musi przechowywać kontekst kryptograficzny dla każdego obsługiwanego obwodu, buforować pakiety oraz zarządzać czasami oczekiwania i ponownym zestawianiem obwodów. W praktyce konieczne jest odgórne ograniczenie liczby równoczesnych obwodów obsługiwanych przez dany przekaźnik. Takie rozwiązanie idealnie współdziała z licznością mikrokontrolerów, które pozwalają na tworzenie o wiele bardziej rozproszonych obwodów. W takim wypadku należałoby wprowadzić też metody pozwalające na nie trzymanie w pamięci większości listy przekaźników, na przykład przez dynamiczne budowanie obwodów. Należy również zaprojektować mechanizmy odrzucania lub opóźniania nowych obwodów przy wyczerpaniu zasobów, aby uniknąć przeciążeń i aby klient zbudował inny obwód. ESP32 dysponuje sprzętowymi akceleratorami AES, SHA, RSA i ECC, co pozwala ograniczyć narzut obliczeniowy zarówno przy tworzeniu obwodu, jak i przy przetwarzaniu pakietów w roli przekaźnika. Konieczne jest jednak rozplanowanie czasowo zadań tak, aby obsługa roli klienta i przekaźnika nie prowadziła do głodzenia pozostałych zadań ani do przekroczenia dopuszczalnych opóźnień. W praktyce oznacza to podział implementacji na niezależne zadania oraz wykorzystanie mechanizmów FreeRTOS dostępnych w ESP-IDF. Przekaźnik ESP32 może być narażony na podstawowe ataki

analizy czasowej i rozmiarowej. Aby się przed tym bronić, należy stosować pakiety o stałym rozmiarze, ruch maskujący, oraz buforowanie i porcjowanie pakietów. Pełna ochrona przed analizą czasową nie jest celem pracy, lecz elementem oceny ograniczeń rozwiązania.

2.5 Podsumowanie analizy wymagań

Przeprowadzona analiza prowadzi do następujących wniosków, które stanowią podstawę dla kolejnych rozdziałów:

- ESP32 będzie pełniło jednocześnie rolę klienta oraz przekaźnika, co oznacza, że implementacja musi obsługiwać zestawianie obwodów, warstwowe szyfrowanie, przekazywanie wiadomości oraz zarządzanie ograniczoną liczbą równoczesnych obwodów po stronie przekaźnika.
- Pełna implementacja protokołu Tor w roli przekaźnika na ESP32 nie jest wykonalna ze względu na ograniczenia pamięciowe. Konieczne jest zaprojektowanie lekkiego, dedykowanego rozwiązania trasowania cebulowego dla sieci testowej złożonej z ESP32, ewentualnie z możliwością integracji klienta z istniejącą siecią Tor w scenariuszu porównawczym.
- Wymagania niefunkcjonalne - zwłaszcza pamięciowe, obliczeniowe i energetyczne - mają charakter nadrzędny i będą determinować wybór konkretnych rozwiązań technicznych w rozdziale projektowym. Kluczowe znaczenie ma ograniczenie liczby równoczesnych obwodów, statyczne zarządzanie pamięcią oraz wykorzystanie sprzętowych akceleratorów kryptograficznych.
- Architektura rozwiązania powinna przewidywać tryb pracy dwurodzajowej - to samo urządzenie ESP32 może być skonfigurowane jako klient, przekaźnik lub węzeł pełniący obie funkcje równocześnie. Wpływa to bezpośrednio na projekt API, model współbieżności oraz sposób testowania wydajności i bezpieczeństwa w Rozdziale 6.

Przyjęte założenia i wynikające z nich decyzje projektowe zostaną rozwinięte w Rozdziale 3 (prototypy teoretyczne) oraz Rozdziale 4 (projekt rozwiązania), gdzie określona zostanie konkretna architektura sieci testowej, API modułu trasowania i przekaźnika oraz metody realizacji trasowania cebulowego w pełnym zakresie.

Rozdział 3

prototypy teorytyczne

Rozdział 4

projekt rozwiązania

Rozdział 5

implementacja

Rozdział 6

testy

Rozdział 7

Podsumowanie